1 Literature Review(weijie)

Recent advances in SLAM (Simultaneous Localization and Mapping) technologies have increased their use in areas such as indoor navigation, mapping, and inspection automation. As new applications for robots continue to develop, achieving reliable and accurate localization while building maps in unknown environments has become a key research focus. Many solutions have been proposed in the form of mature SLAM systems and implementation frameworks, such as ORB-SLAM2 [1], RTAB-Map [2], and Cartographer [3], which use multi-sensor fusion methods to improve environmental perception and localization performance.

Current mainstream solutions for SLAM using perception involve accommadating multiple types of sensors, which include, but are not limited to, LiDAR, ToF depth cameras, structured light sensors, stereo or multi-camera vision systems, and inertial measurement units (IMUs). LiDAR has some of the highest accuracy and reliability for mapping in difficult environments, while visual sensors carry several advantages including being less costly and providing greater detail about their surroundings, as well as being able to provide stable estimates of people's poses over shorter times. By fusing data from various types of sensors, systems can provide strong localization and mapping consistency more regularly, despite differences between their respective environments. The consequent impact of these attributes makes multi-sensor fusion an essential development area within SLAM-based technologies.

Although the performance of these approaches is promising as shown in Table 1, mainstream SLAM systems typically have a much more complicated hardware configuration and require a large amount of computational resources. As a result, they can create a number of problems. First, generally SLAM systems will require LiDAR that provide very high levels of accuracy and a high-performance computer to compute, which means an extremely high cost and thus difficult to deploy on a large scale. Second, given that most SLAM systems utilize many different sensors that are supported by various structures to hold them in place, the physical size of the complete system is considerably larger than if only a single sensor were used, which limits them from being able to fit into narrow areas or work within extremely cluttered environments. Lastly, to create a single, cohesive picture from multiple sensors will require calibrating, synchronizing, and fusing the data from all the sensors, adding additional complexity to the development of the system and also increasing the cost for future maintenance of the system after it has been created. These issues become even more pronounced in certain situations:

- In rapid scanning tasks, users care more about obtaining the environment structure at a lower cost rather than high-precision mapping
- In confined spaces, the robot's size becomes the main limitation
- In large-scale or high-frequency inspection tasks, high-cost platforms will also limit the system's scalability

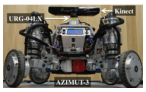
Project Name	Project images	Project approach	Project advantages	Project disadvantages
Clearpath Husky UGV	[9]	Multi-sensor fusion + SLAM mapping	• High precision • Strong stability • Mature ROS ecosystem	• High cost • Complex system
Intel RealSense D435i + RTAB-Map	 [10]	Visual SLAM (RGB-D) + RTAB-Map mapping	• Low cost • Information-rich • Good indoor performance • RTAB-Map is mature	• Sensitive to lighting • Limited depth range • Less accurate than LiDAR SLAM
Boston Dynamics Spot	[11]	LiDAR SLAM + quadruped robot platform	• Strong terrain adaptability • Strong 3D mapping capability • Industrial-grade stability	• Very high cost • Closed system • Requires large computing resources

Table 1: Comparison of representative SLAM-based robotic platforms

Therefore, how to reduce system cost while ensuring basic perception capability has become an important research direction.

In response to the above issues, existing research has begun to explore low-cost SLAM solutions. For example, monocular vision-based SLAM methods achieve environmental mapping using only a single camera, thereby significantly reducing hardware costs. At the same time, with the development of deep learning, image-based depth estimation methods have provided new technical pathways for monocular SLAM. However, such methods still have certain limitations in terms of accuracy and stability, especially facing challenges in scale recovery and handling dynamic environments.

On this basis, this paper proposes a SLAM solution for low-cost scenarios. The method uses a monocular camera as the main sensor to reduce system hardware cost, and combines learning methods to obtain spatial information of the environment. While ensuring basic perception capability, this design achieves a good balance between cost and complexity.

In order to improve user experience and environmental coverage of the system, this paper presents a multi-robot cooperative mechanism known as multi-agent team coordination (MATC). Multiple small robots may perform parallel data collection, thus increasing the overall speed and providing greater flexibility and scalability to the system when used in very complex or large dynamic environments, by performing data collection using swarm methods.

In addition, regarding the computational resource requirements of learn-

ing methods, the system adopts a centralized computing architecture, where computation-intensive tasks are executed on high-performance devices, while the robots are mainly responsible for data collection and motion control. This design improves the overall system efficiency without significantly increasing the cost of individual units.

Overall, the method in this paper combines monocular vision, swarm-style multi-robot collaboration, and a centralized computing architecture. It reduces system cost while improving scanning efficiency and system scalability, and provides a feasible approach for implementing low-cost SLAM systems.

2 Theoretical, Mathematical and Scientific Background(weijie)

2.1 Map Representation and Frontier Definition

This project uses a two-dimensional occupancy grid to represent the environment. Each grid cell is defined as:

$$m_{i,j} \in \{-1, 0, 1\}$$

Among them, -1: UNKNOWN, 0: FREE, 1: OCC.

On this basis, define frontier as:

$$\mathit{Frontier} = \{(i,j) \mid m_{i,j} = \mathit{FREE},\ \exists\ \mathit{neighbour} = \mathit{UNKNOWN}\}$$

A frontier is defined as a cell that is traversable itself and is adjacent to at least one unknown grid cell.

2.2 Robot Motion and Coordinate Model

The robot moves in continuous space, and its state is represented as:

$$s = (x, z, \theta)$$

Where θ is the orientation angle, and the direction of motion is determined by:

$$\Delta r = \cos(\theta), \quad \Delta c = \sin(\theta)$$

Update the rule:

$$r_{t+1} = r_t + \Delta r \cdot d, \quad c_{t+1} = c_t + \Delta c \cdot d$$

Where d represents the distance moved.

2.3 Path Planning Methods (A^*)

Path planning uses the A^* algorithm, and its objective function is:

$$f(n) = g(n) + h(n)$$

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

To account for environmental risk, incorporate map cost into the path cost:

$$g(n) = g(n-1) + 1 + \lambda \cdot cost(n)$$

This not only finds a shorter path, but also avoids passing through high-risk areas, making the planning results more aligned with the real environment.

2.4 Collision Detection Model

The robot is modeled as a circle, and its feasible region satisfies:

$$(x - x_0)^2 + (y - y_0)^2 \leq r^2$$

r is the robot radius. During pathfinding, the movement path is continuously sampled, and each point is checked for collisions with obstacles. Only when all sampled points along the entire path are in traversable areas is the path considered safe.

2.5 Perception Models and Information Acquisition

The robot uses an FOV model for perception:

$$\theta \in \left[\theta_0 - \frac{FOV}{2},\ \theta_0 + \frac{FOV}{2}\right]$$

The number of visible unknown areas within the field of view is:

$$U = \sum 1_{cell = UNKNOWN}$$

This guides the robot to prioritize exploring directions with more unknown areas.

2.6 Observation Overlap Constraint

To ensure continuous observations can be fused, introduce overlap constraints:

$$Overlap = \frac{FOV - |\Delta\theta|}{FOV}$$

When the orientation change is too large, the overlapping area will decrease, thereby affecting the map stitching result. Therefore, constraints are set:

$$Overlap \geq min_overlap_ratio$$

This constraint is used to limit the robot's rotation at each step, ensuring sufficient overlap between adjacent observations.

2.7 Single-robot Decision-making Model

The robot selects targets through a composite scoring function:

$$Score = Path_{length} + Yaw_{penalty} + Cost_{penalty} + Overlap_{penalty} - Unknown_{gain}$$

Where:

- $Path_{length}$: the path length to the target.
- $Yaw_{penalty}$: the cost caused by the turning angle.
- $Cost_{penalty}$: the risk of the area the path passes through.
- $Unknown_{gain}$: the amount of unexplored area that can be explored.

The robot selects the target with the lowest score as its current movement direction. Because path length, turning cost, and environmental risk are all cost terms, and unknown areas are benefit terms, the minimum score is the optimal solution.

2.8 Multi-robot Collision Avoidance

In multi-robot systems, it is necessary to avoid conflicts between robots, so a multi-layer collision avoidance mechanism is adopted:

First, position constraints:

$$(x_i, y_i) \neq (x_j, y_j)$$

Secondly, a safe distance:

$$d(r_i, r_j) > d_{\min}$$

2.9 Field-of-view Constraints and Cooperative Avoidance

When there is visual interaction between robots:

$$r_j \in FOV_i$$

When the corresponding area is marked as impassable:

$$M_{plan} = M \setminus Forbidden$$

3 Methods, Setups and Software Implementation(weijie)

3.1 Path-plan Overview

This path planning module enables the robot to start from an unknown environment, gradually expand the known area, and use detected frontier regions as exploration targets until no frontiers remain in the map.

3.2 Perception and Map Update

The robot perceives the environment using an FOV model. Within its current facing direction, it uses multiple rays to obtain information about obstacles and traversable areas, and updates the occupancy grid map. At the same time, it performs local updates within a certain range around the robot to improve near-range perception accuracy. The hardware parameters are: camera field of view: 45°, perception range 8 cells, grid size: 5 cm.

To simulate perception delay in real systems, a MapBuffer is introduced in the code. In path planning, the map from the previous frame is used for decision-making, which reflects the time gap between perception and processing in real systems.

3.3 Path Planning and Decision Making

The input to the path planning module consists of the cost-map in `.npy` format provided by the mapping module, together with the robot's current world pose represented by (x, z, yaw). The planner first converts the world pose into the corresponding grid position on the current map, and then uses the map cost and free/occupied information to generate the next motion decision.

The output of the planner is not a complete global trajectory, but an executable control sequence in the form

$$\{(c_1, p_1), (c_2, p_2), \ldots\}$$

where the first element of each pair is the command type and the second element is its parameter. In this system, 0 denotes stop, 1 denotes forward motion, 2 denotes backward motion, 3 denotes left rotation, and 4 denotes right rotation. The corresponding parameter is either the forward distance in cm or the rotation angle in degrees. This output format allows the planning result to be directly used by the robot control module as shown in Fig. 1.

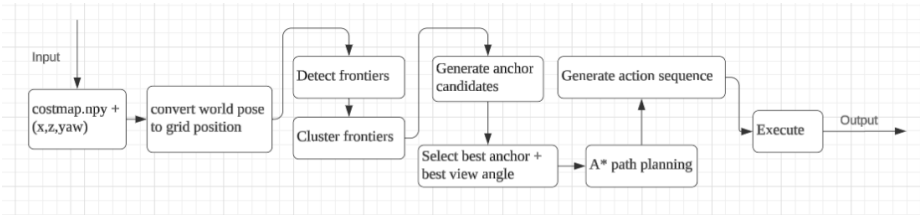


Figure 1: Decision-making pipeline of the robot for each planning step

The system detects regions near the edges of the current map (either FREE or adjacent to UNKNOWN) using the method described above, and then connects them into clusters based on 8-neighborhood connectivity. After the clusters are formed, various possible anchors are generated for each frontier region to use as potential observation locations. The primary planningalgorithm is a

cost-conscious frontier-based A^* form of a search algorithm. Frontiers are initially obtained from an occupancy grid, and after that, we generate potential observation anchors around these frontier areas. For every anchor, the system considers various potential angles of vision and scores those potential angles of vision based on various criteria: path length, turning cost, map cost, and anticipated information gain. After the system selects the best anchor, it computes the best feasible path for the robot from its current location to the anchor using a modified version of the A*search algorithm.

The system will simulate the effect of observing from each of the generated anchors with respect to different angles of observation. Each of the anchors will then be evaluated by determining the path length, turning cost, and amount of information gained for each simulation. Subsequently, the optimum anchor for each frontier will be selected based on the results of the evaluations listed. This provides a method for the robot to observe its surroundings while at the same time, minimizing how far it must travel to get to the frontier.

Once the optimum anchor has been determined, the A^* algorithm will be utilized to compute a valid path between the robot's current position and the optimum anchor location. The robot will follow the computed path to the anchor location and perform its observation at the anchor; thus establishing a balance between traveling the shortest path and acquiring the greatest amount of information.

3.4 Motion and Constraints

This robotic system has a single-step strategy with maximum movement (forward) of up to 2 cells (10 cm) in distance for each step, maximum rotation of 15°, 1 cell's worth of inflated obstacles (5 cm), and has an observation overlap constraint that requires at least 0.35 (min overlap) in order to support map stitching by providing enough overlapping area between the two observations.

At every step, the robot's computer system will evaluate how to balance (with respect to time) between the actions of "scanning first" or "moving first" at the same time. Hence, determining the amount of information required to obtain is equal to the amount required to move forward in the same time period.

3.5 Multi-Robot Coordination

The implemented system consists of a fully operational three-robot exploration framework, as illustrated in Fig. 2. All robots contribute to a shared global occupancy grid while performing sensing and decision-making independently. First, a target exclusion strategy [4] is used to prevent multiple robots from selecting the same frontier region, thereby reducing repeated exploration. Second, collision avoidance rules [5] are introduced during movement, including prohibiting robots from occupying the same position and maintaining a minimum safe distance.

In addition, a priority-based conflict resolution mechanism [6] is introduced. When there is a potential conflict between robots, the robot with higher priority

Figure 2: Multi-robot exploration process over time

moves first, while the others remain stationary. The priority in the system is determined by the robot ID.

Furthermore, the system introduces field-of-view constraints [7]. When a robot may enter the visual field of another robot, that area is marked as a non-traversable region to avoid visual interference.

Finally, a phased deployment strategy [8] is adopted, allowing robots to gradually enter the environment, reducing congestion in the initial stage and improving system stability. Specifically, Robot 0 starts first, and Robot $i+1$ is only deployed after Robot i has moved sufficiently far from the initial position. This reduces congestion during the initial exploration stage and improves overall system stability.

3.6 Design Justification and Comparison with Existing Methods

The Path-plan part does not use complex optimization algorithms or learning methods. Instead, a cost-aware A^* strategy is adopted, as it provides a good balance between computational efficiency and planning reliability under the current system setting.

To further justify this choice, several commonly used planning strategies were compared, including A^*-based frontier exploration, random walk, DWA-like methods, VFH-like methods, and D^* Lite. The comparison was conducted over multiple maps using both exploration efficiency (measured by the number of steps) and coverage rate. As shown in Fig. 3, the A^*-based nearest frontier method achieves near-complete coverage (approximately 99.96%) while requiring significantly fewer steps than most baseline methods. In contrast, methods such as random walk and DWA-like approaches either require substantially more steps or fail to achieve sufficient coverage.

In addition, computational performance was evaluated in dynamic scenarios. The results show that A^* requires significantly less total planning time than D^* Lite, while maintaining comparable exploration performance,as illustrated in Fig. 4. This is because the current system relies on step-by-step local replanning, where the overhead of incremental algorithms such as D^* Lite does not

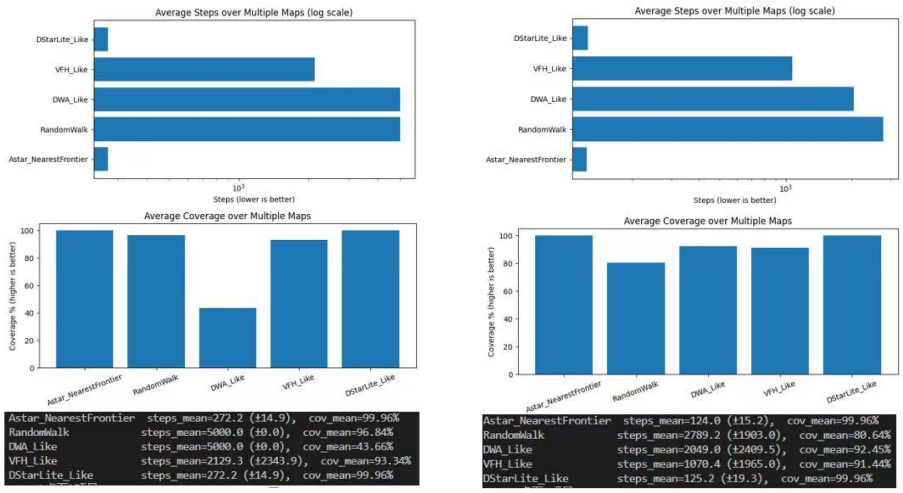


Figure 3: Comparison of exploration performance across different planning methods.

provide clear advantages. Therefore, considering both exploration effectiveness and computational cost, A^* is selected as the core path planning algorithm in this work. The Path-plan part does not use complex optimization algorithms

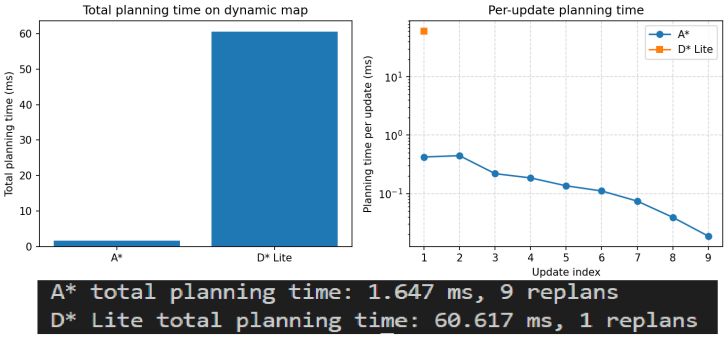


Figure 4: Comparison of total planning time and per-update planning time between A* and D* Lite in dynamic environments.

or learning methods. This choice is mainly based on considerations of computational complexity and system stability. Compared with global optimization or distributed methods, the current rule-based method is simple to implement, easy to interpret, and can run stably with limited computational resources.

In addition, this part introduces field-of-view constraints and a perception delay model, making the system closer to real-world application scenarios. Although these designs increase the complexity of path planning, they better reflect perception limits in actual systems, thus improving the practical signifi-

cance of the project.

Compared with existing multi-robot exploration methods, the approach in this part may not match advanced methods in terms of theoretical optimality, but it has clear advantages in implementation complexity, stability, and reproducibility. At the same time, field-of-view constraints and delay mechanisms are rarely considered in traditional methods, giving this part a certain level of innovation in simulating real environments.

4 Future work(weijie)

First, in the current method, target selection is based on a scoring mechanism of frontier clusters and anchors, and decisions are made by combining path length, turning cost, and information gain. Although this method has achieved good exploration efficiency in experiments, in complex environments it may still encounter repeated paths and local optimum issues. In the future, the target scoring function can be further optimized, for example by introducing global information distribution or historical visitation information, enabling the system to proactively avoid explored areas, thereby reducing repeated paths and improving overall exploration efficiency.

Second, the current method for path planning is to use the A^* algorithm with cost awareness and to replan at each time step. It is both an easy and stable method, but it has some redundancy in terms of computation due to its operations in dynamic environments or due to the fact that maps may be updated so frequently. As such, one potential way to address the aforementioned redundancies is to combine incremental planning methods for path planning to minimise redundant computations, while maintaining the quality of paths produced; the aim being to improve the responsiveness of the system in situations that are dynamic.

Third, the system has also used the FOV approach and the concept of observation overlap in the perception and decision-making process in order to guarantee that neighboring images contain sufficient overlapping parts for map construction. However, at present, there are rather rigid restrictions on the rotation angles of viewpoints and observation orientations. In the future, the overlap scheme should be made adaptive to dynamically change viewpoints and observation orientations according to environmental complexity.

Finally, in multi-robot systems, current methods mainly avoid conflicts through goal exclusion, priority mechanisms, and field-of-view constraints. Although these can sometimes ensure stable system operation, overall coordination is still relatively local. In the future, more advanced coordination mechanisms can be introduced, such as global optimization methods based on task allocation, allowing different robots to divide work over a larger range, thereby further reducing redundant exploration and improving overall efficiency.

5 Appendix(weijie)

The main implementation file for this submodule is `planner.py`. This file contains the core functions used in this work, including frontier detection, anchor selection, path planning based on the A^* algorithm, and multi-robot coordination. It is the main code file used to generate the planning results and simulation outputs presented in this report.

The writing process related to this part of the report is documented in the following repository:

`https://github.com/ntjj6/final-report`

This repository contains the writing history of this section, including early drafts, revised versions, and the final written content used in the report. AI tools were used only for grammar checking, correction of language errors, and refinement of wording. The design of the algorithm, implementation details, experimental setup, and the core technical content were independently developed by the author.

References

[1] R. Mur-Artal and J. D. Tardós, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE Transactions on Robotics*, vol. 33, pp. 1255–1262, Oct 2017.

[2] M. Labbé and F. Michaud, "Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

[3] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2d lidar slam," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, (Stockholm, Sweden), pp. 1271–1278, 2016.

[4] M. Zhu, Z. Liu, W. Li, S. Wang, and Q. Zhang, "Improved wavefront frontier detection-utility value task allocation for multi-robot collaborative environmental exploration," *Automation in Construction*, 2026.

[5] A. A. Amanatiadis and S. A. Chatzichristofis, "A multi-objective exploration strategy for mobile robots under operational constraints," in *Proceedings of the IEEE International Conference*, 2013.

[6] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, and D. Sadigh, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Autonomous Robots*, vol. 46, no. 1, pp. 1–21, 2022.

[7] M. Catellani and L. Sabattini, "Distributed control of a limited angular field-of-view multi-robot system in communication-denied scenarios: A probabilistic approach," *IEEE Robotics and Automation Letters*, vol. 8, no. 12, pp. 1–8, 2023.

[8] G. Best, R. Garg, and J. Keller, "Multi-robot, multi-sensor exploration of multifarious environments with full mission aerial autonomy," *The International Journal of Robotics Research*, 2024.